

// ok.

SwiftBSV is a pure-Swift SDK for Bitcoin SV. It powers the wallet inside Henceforth and ships as a standalone Swift Package for any iOS or macOS app that needs Bitcoin keys, transactions, scripts, or SPV verification.

1.1 Links to Source Code

github.com/henryhudson/SwiftBSV

Add it to a Swift package via the dependency stanza shown in the Integration chapter – one line in Package.swift or one menu click in Xcode.

1.2 Status

- **Swift tools 5.9+**, builds cleanly under **Swift 6** with strict concurrency.
- **Sendable** throughout the public surface – pass values across actor boundaries without @unchecked or @preconcurrency workarounds.
- **No deprecated dependencies.** CryptoSwift 1.9.x and Boilertalk’s secp256k1 binding are the only third-party packages.
- **CI green** on macOS 15 + Xcode 16 via GitHub Actions on every push to main.

1.3 What This Book Covers

The chapters that follow walk every public type in the SDK from the bottom up:

- Cryptographic primitives (Point, BInt, Crypto)
- Keys and addresses (PrivateKey, PublicKey, Address, WIF)
- HD wallets (Bip39, Bip32) and Type42 (BRC-42) invoice-numbered derivation
- Signatures (ECDSA, sighash, BSM) and ECIES encryption (BIE1 wire format)
- Scripts and the full BSV opcode set

- Transactions and the fluent TxBuilder
- Simplified Payment Verification (block headers + Merkle proofs)
- SwiftPM integration and contributing

Every chapter includes runnable examples in dark terminal panels and notebox callouts for the safety invariants you cannot opt out of.

2.1 What SwiftBSV Is

- **Idiomatic Swift.** Value types where they make sense, classes only where mutation lifecycle demands it (TxBuilder, Bip39). No NS prefixes leaking through, no implicit globals, no completion-handler closures where async/await would do.
- **Compile-time safe.** Optionals where failure is part of the contract (parsing a malformed WIF, decoding a truncated DER signature). Throwing functions where the caller must handle it. No try!, no fatalError in the production path.
- **Wire-compatible with the rest of BSV.** ECIES emits BIE1-format ciphertext readable by @bsv/sdk. BSM signatures verify against any other Bitcoin Signed Message implementation. Transaction binary matches whatever a node will accept.
- **Small surface, deep semantics.** Every public type has a single clear responsibility. The whole library fits in your head once you've read this book.

2.2 What SwiftBSV Is Not

- Not a wallet. Wallet UX, key storage, address discovery, sync strategy – those live one layer up. SwiftBSV gives you the primitives; Henceforth shows one way to compose them.
- Not a network client. Broadcasting, fee quoting, UTXO fetching, Merkle-proof retrieval – all out of scope. SwiftBSV validates a Merkle proof; fetching one over HTTPS is your problem.
- Not a token/ordinal SDK. BSV-20, BSV-21, 1Sat Ordinals, BAP, MAP – all build on top of the primitives here. SwiftBSV stays at the protocol layer.

2.3 Future

- Direct support for BEEF (BRC-62) and BUMP (BRC-74) construction, not just verification.
- Streaming Merkle-tree construction for very large blocks.

- More extensive opcode tests – the Hayes-style suite that covers Henceforth’s FORTH side has no exact analogue here yet.
- A formal benchmark suite tracking ECDSA verification and SHA-256 throughput across iPhone generations.

3.1 Will Townsend – Original SwiftBSV

SwiftBSV was originally written by Will Townsend at [wtsnz/SwiftBSV](https://github.com/wtsnz/SwiftBSV) as a Swift port of the moneybutton/bsv JavaScript library, mashing together ideas from several earlier Swift Bitcoin projects (Yenom's BitcoinKit, Kevin Vitale's WalletKit, yuzushioh's HDWalletKit). The current fork at [henryhudson/SwiftBSV](https://github.com/henryhudson/SwiftBSV) continues from that foundation: BSV-specific protocol decisions (restored opcodes, FORKID sighash, no segwit), Swift 6 strict concurrency, defensive Data slicing throughout the public surface, ECIES BIE1 wire compatibility, and a SPV verification module. Will's authorship of the BIP-32/39 implementation, the BInt big-integer wrapper, the Base58Check encoder, and the original TxBuilder fluent API is unchanged in this fork – the structural debt is large and acknowledged.

3.2 Boilertalk's secp256k1.swift

Boilertalk's [secp256k1.swift](#) wraps Pieter Wuille's reference C library (libsecp256k1) for Swift. SwiftBSV uses it for every elliptic-curve operation: ECDSA sign and verify, public-key tweak (Type42), compact signature recovery (BSM). When you see `Secp256k1.signCompact` or `publicKeyTweakMul` in the source, you're calling through Boilertalk's binding into the C library that underlies Bitcoin Core itself.

3.3 CryptoSwift

[Marcin Krzyżanowski's CryptoSwift](#) provides SHA-256, SHA-512, RIPEMD-160, AES-CBC, HMAC, and PBKDF2. Apple's CryptoKit covers most modern needs but lacks RIPEMD-160 (used in Hash160 for address derivation) and AES-CBC (used in ECIES BIE1). CryptoSwift fills both gaps in a Swift-native way.

3.4 Bitcoin White Paper

The reference for everything in this book. Section 8 in particular – Simplified Payment Verification – defines the trust model implemented in the SPV chapter.

[Bitcoin White Paper](#)

3.5 Henceforth

The companion book [Henceforth](#) documents the iOS app that consumes this SDK. SwiftBSV’s design choices are calibrated for that consumer: every type that crosses an @MainActor boundary is Sendable; every initializer that touches the network is absent (because Henceforth’s networking layer owns that); every value type is Equatable and Hashable where the protocol allows it because Henceforth’s UI uses them in ForEach and Set routinely.

Layering. The types in this chapter are the foundation everything else stands on. You rarely construct them directly – PrivateKey hides BInt, PublicKey hides Point, Hash160 is called for you by Address. They’re documented here so when you see them in a stack trace or a generic constraint, you know what they mean.

4.1 BInt – Big Integer

Bitcoin keys are 256-bit integers. Swift’s Int tops out at 64 bits, so SwiftBSV ships its own arbitrary-precision integer type, BInt, modeled on the conventional sign-magnitude representation. It conforms to Comparable, Equatable, Hashable, and the standard arithmetic operators.

BInt – Constructing 256-bit values

```
let n = BInt("01abcdef", radix: 16)! // hex literal
let m = BInt(decimalString: "1000000000000000000000")!
let sum = n + m                       // operator overloading
let neg = -m                          // unary minus

if n < m { print("n is smaller") }
```

You almost never write BInt directly in app code. The library uses it internally for the curve order N, the integer form of a private key, and the components of an ECDSA signature.

BInt is @unchecked Sendable. The wrapped storage is internally a Limbs array of UInt64, mutated only inside arithmetic operators that produce new BInt values. There is no shared mutable state; the @unchecked is paying for the limitation that Array<UInt64> is not itself Sendable as of Swift 6, even though the way BInt uses it is.

4.2 Point – A Point on secp256k1

Point represents an (x, y) coordinate on the secp256k1 curve. It is the mathematical content of a public key.

Point – Internals you usually don't touch

```

let pubkey: PublicKey = privateKey.publicKey
let point: Point = pubkey.point          // x: BInt, y: BInt

// Compressed serialization (33 bytes)
let compressed: Data = pubkey.toBuffer()

// DER serialization (33 or 65 bytes)
let der: Data = pubkey.toDer(compressed: true)

```

You only handle Point directly in two places: when you write your own opcode-level script that pushes a curve point, or when you implement a multi-signature scheme by hand. For ordinary signing, encryption, and address derivation, PublicKey is the right type.

4.3 Crypto – Hash and HMAC Helpers

The Crypto enum bundles the hash functions Bitcoin uses, plus an HMAC helper. Static functions only – no state, no instance.

Crypto – Hash functions

```

let bytes: Data = "hello".data(using: .utf8)!

let sha256: Data = Crypto.sha256(bytes)           // 32 bytes
let sha256d: Data = Crypto.sha256sha256(bytes)    // 32 bytes
(BTC standard)
let hash160: Data = Crypto.sha256ripemd160(bytes) // 20 bytes
(address hash)
let sha512: Data = Crypto.sha512(bytes)           // 64 bytes

let key: Data = "secret".data(using: .utf8)!
let mac: Data = Crypto.hmacsha256(key: key, data: bytes)
let mac2: Data = Crypto.hmacsha512(key: key, data: bytes)

```

Why double SHA-256? Bitcoin double-hashes nearly everything: transaction IDs, block header hashes, the message-digest step of ECDSA. The doubling defends against length-extension attacks on Merkle-Damgård hash functions and gives a second pass of avalanche even if a structural weakness in the inner hash is later discovered. Crypto.sha256sha256 encapsulates this so you don't accidentally single-hash where the protocol calls for double-hashing.

4.3.1 verifySigData

A combined hash-and-verify helper:

Crypto.verifySigData

```
let data: Data = ...           // arbitrary message bytes
let sig: Data = ...           // DER-encoded signature
let pub: PublicKey = ...

let ok: Bool = Crypto.verifySigData(
  for: data,
  signature: sig,
  publicKey: pub
)
```

The function double-SHA-256s the input before verifying – matching Bitcoin’s checksig semantics – so you pass the raw payload, not a pre-computed hash.

5.1 Network

Everything in SwiftBSV that has a wire-format byte representation carries a `Network` enum so the same code can produce mainnet and testnet artifacts:

Network

```
public enum Network: Sendable {  
    case mainnet  
    case testnet  
}
```

The default everywhere is `.mainnet`. Pass `.testnet` explicitly when you mean it.

5.2 PrivateKey

A `PrivateKey` wraps a 256-bit secret integer (`BInt`), a network tag, and a compression flag.

PrivateKey - Constructors

```
// Random key for the given network  
let key = PrivateKey(network: .mainnet)  
  
// From 32-byte raw seed  
let key2 = PrivateKey(data: bytes, network: .mainnet)  
  
// From WIF (Wallet Import Format) -- fails on bad checksum or  
// wrong network byte  
guard let key3 = PrivateKey(wif: "L1aW4...") else { return }  
  
// Properties  
let pub: PublicKey = key.publicKey // derived on-demand  
let addr: Address = key.address // P2PKH for the network  
let raw: Data = key.data // 32 raw bytes
```

5.2.1 WIF Round-trip

Wallet Import Format is the standard textual encoding for private keys – a Base58Check string with a network-discriminating version byte:

PrivateKey – WIF

```
let key = PrivateKey(network: .mainnet)
let wif: String = key.toWif()           // "L1aW4...", 52 chars
    compressed
let restored = PrivateKey(wif: wif)     // returns Optional
```

Compression flag survives WIF. A WIF for a compressed key has a trailing 0x01 byte before the checksum is computed. Round-tripping through WIF preserves whether the public key should serialize as 33 bytes or 65 bytes. Don't lose track of the flag – a "correct" key produces a different address depending on its compression bit.

5.3 PublicKey

A `PublicKey` is the curve point a private key derives to, plus its compression flag.

PublicKey – Construction & serialization

```
// From hex (33 or 65 bytes)
guard let pk = PublicKey(hex: "02a3b1c...") else { return }

// From DER bytes (33 / 65)
guard let pk2 = PublicKey(fromDer: derBytes, isStrict: true) else {
    return
}

// Serialize back
let derCompressed: Data = pk.toDer(compressed: true)
let derUncompressed: Data = pk.toDer(compressed: false)
let raw: Data = pk.toBuffer() // 33 or 65 bytes

// Address (P2PKH)
let addr: Address = pk.address
```

5.3.1 Strict vs Lax DER Parsing

The `isStrict` flag on `init(fromDer:isStrict:)` controls whether the parser rejects malformed prefixes. Production callers should always pass `true`; the lax mode exists only for compatibility with historical malleable encodings and should not be used for newly received data.

Defensive Data slicing. `init(fromDer:)` explicitly materializes its input via `let buffer = Data(buffer)` before subscripting. A `Data` slice in Swift preserves its parent's indices – subscripting at `[0]` crashes if the slice was taken from a non-zero offset. SwiftBSV defends against this at every `init?` that takes a `Data` parameter, because callers in real apps pass slices all the time without realizing it.

5.4 Address

`Address` is a P2PKH (Pay-to-Public-Key-Hash) destination. Construction is total – pass any of the upstream key types and you get an address back:

Address – Construction

```
let priv = PrivateKey(network: .mainnet)
let addr1 = Address(priv)                // from PrivateKey
let addr2 = Address(priv.publicKey)      // from PublicKey
let addr3 = Address(bip32, network: .mainnet) // from a Bip32 node

// Parse a Base58Check string
guard let addr4 = Address(fromString: "1A1zP1...", network: .mainnet)
else { return }
```

5.4.1 Output Script

To pay an address, you need its locking script:

Address – Locking script

```
let script: Script = address.toTxOutputScript()
// OP_DUP OP_HASH160 <20 bytes> OP_EQUALVERIFY OP_CHECKSIG
```

`Address` produces a standard P2PKH script. For non-standard outputs (P2SH, multisig, `OP_RETURN`), build the `Script` directly – see the `Scripts` chapter.

6.1 Bip39 – Mnemonic Phrases

A 12-word or 24-word phrase from the BIP-39 word list. The phrase encodes 128 or 256 bits of entropy plus a checksum, and deterministically derives a 64-byte seed.

Bip39 – Generating a new mnemonic

```
// Standard 24-word phrase (256 bits of entropy)
let phrase: String = Bip39.create(strength: .high)

// 12-word phrase (128 bits of entropy)
let short: String = Bip39.create(strength: .low)

// From specific entropy (e.g. dice-rolled 32 bytes)
let custom: String = Bip39.create(entropy: myEntropy)

// Convert phrase to seed (with optional passphrase)
let seed: Data = Bip39.createSeed(
    mnemonic: phrase,
    withPassphrase: ""
)
```

The **passphrase argument is real**. BIP-39 specifies an optional user passphrase that mixes into PBKDF2 alongside the mnemonic. Two wallets seeded from the same 24-word phrase but different passphrases derive entirely different key trees. The Henceforth wallet does not currently expose this field – the empty string is used – but the library exposes it for tooling that does.

6.2 Bip32 – Hierarchical Deterministic Keys

A Bip32 node represents a single point in the HD key tree. From the seed produced by Bip39, you derive a master node, then descend by path:

Bip32 – Master and derivation

```
let seed = Bip39.createSeed(mnemonic: phrase)
let master = Bip32(seed: seed, network: .mainnet)

// Derive m/44'/236'/0'/0/0 (BIP-44 BSV receive address index 0)
guard let node = master.derivedKey(path: "m/44'/236'/0'/0/0")
else { return }

let priv: PrivateKey? = node.privateKey
let pub: PublicKey    = node.publicKey
let addr: Address     = node.address
```

6.2.1 DerivationNode

Inside the path string, each step is encoded as a DerivationNode:

Bip32 – DerivationNode

```
public enum DerivationNode {
    case hardened(UInt32) // 0x80000000 | index
    case notHardened(UInt32)
}

// Equivalent to "m/44'/236'/0'"
let n1 = DerivationNode.hardened(44)
let n2 = DerivationNode.hardened(236)
let n3 = DerivationNode.hardened(0)
```

Hardened derivation breaks the public-key-only chain – you cannot derive hardened children from an extended public key alone. This is intentional: it isolates account-level keys from leaf-level exposure.

6.2.2 Public-Only Derivation

For watching-only wallets, you can hand someone an extended public key and let them derive child addresses without ever seeing the private side:

Bip32 – Public-only mode

```
let masterPub = master.toPublic()
let xpub: String = masterPub.toString()

// On the receiving device:
guard let pubMaster = Bip32(string: xpub) else { return }
guard let leaf = pubMaster.derivedKey(path: "m/0/0")
else { return }
// leaf.privateKey == nil, leaf.publicKey is valid
```


Path syntax. `derivedKey(path:)` accepts the conventional `m/<index>['/']` format. Hardened children use `'` (apostrophe). Empty path `m` returns the receiver. Invalid path strings return `nil` – always unwrap.

6.2.3 Wire Format

The Base58Check serialization (`xprv...`, `xpub...`) round-trips through `toString()` / `init?(string:)` and matches the BIP-32 specification, byte for byte. SwiftBSV-produced `xprv` can be loaded by any standards-compliant BIP-32 implementation.

Type42 is BRC-42. The protocol is specified in **BRC-42**. SwiftBSV implements the full set: derive private keys (counterparty needs your private key + their public key), derive public keys (counterparty needs only public keys), self-derivation, and ECDH shared-secret extraction.

Type42 generates a fresh keypair per invoice number, mixing together one party's private key, the other party's public key, and a string. Two parties who share their public keys offline can independently derive the same address for any invoice number, without ever exchanging private material.

7.1 The Three Derivation Modes

Type42 - derivePrivateKey

```
// You have your private key. Counterparty has their public key.  
// Both of you can compute the same shared invoice key.  
let derived: PrivateKey? = Type42.derivePrivateKey(  
    privateKey: myPrivateKey,  
    counterpartyPublicKey: theirPublicKey,  
    invoiceNumber: "invoice-2026-01-001"  
)
```

Type42 - derivePublicKey

```
// Both parties have only public keys. Useful for watch-only  
// wallets that prepare invoices without holding the spending key.  
let derivedPub: PublicKey? = Type42.derivePublicKey(  
    publicKey: myPublicKey,  
    counterpartyPublicKey: theirPublicKey,  
    invoiceNumber: "invoice-2026-01-001"  
)
```

Type42 - deriveSelf

```
// Self-derivation -- counterparty == self. Used for fresh
// receive addresses inside a single wallet without coordinating
// with anyone.
let receive: PrivateKey? = Type42.deriveSelf(
    privateKey: masterKey,
    invoiceNumber: "wallet/0/recv/42"
)
```

7.2 Henceforth Path Conventions

The wallet uses Type42 invoice numbers as a flat namespace:

- wallet/0/recv/N – receive addresses (was m/44'/236'/0'/0/N in BIP-44)
- wallet/0/change/N – change addresses
- paymail/<handle> – per-paymail keys
- bap/<id> – BAP identity attestation keys

The convenience wrappers `deriveChangeKey`, `derivePaymentKey`, `deriveEncryptionKey`, and `deriveBAPIdentity` build the appropriate invoice-number string and call through to `derivePrivateKey`.

7.3 ECDH Shared Secret

The mathematical operation underpinning Type42 – and ECIES, BRC-103 auth, and any other protocol that needs a shared session key – is ECDH:

Type42 - ecdhSharedSecret

```
let secret: [UInt8]? = Type42.ecdhSharedSecret(
    privateKey: myPrivateKey,
    publicKey: theirPublicKey
)
// 33 bytes -- compressed serialization of the shared point
```

The result is the compressed serialization of the curve point obtained by multiplying the counterparty's public key by your private key. Symmetric: they get the same point by multiplying your public key by their private key.

Don't use the raw shared point as a key. The compressed point has structure – the leading byte is parity, leaving only 256 bits of variable material. For symmetric

encryption you should run it through a KDF (BSV uses SHA-512 of the X coordinate to derive an IV, an AES key, and an HMAC key – see the ECIES chapter). Treating the raw shared point as an AES key is a mistake.

8.1 Signature – ECDSA Values

A Signature bundles (r, s), the sighash type, and a recovery hint. It encodes to two distinct wire formats depending on context:

- **DER** for general ECDSA – variable-length, 70–72 bytes typically.
- **Tx format** for transaction signatures – DER plus a single-byte sighash flag suffix.

Signature – Construction & encoding

```
let key = PrivateKey(network: .mainnet)
let messageHash = Crypto.sha256sha256(message)

// Sign produces a Signature value
let sig = ECDSA.sign(
    hash: messageHash,
    privateKey: key
)

// DER bytes -- for verification by libsecp256k1, openssl, etc.
let der: Data = sig.toDer()

// Transaction format -- DER + sighash byte for OP_CHECKSIG
let txFormat: Data = sig.toTxFormat()

// Verify
let ok = ECDSA.verify(
    hash: messageHash,
    signature: sig,
    publicKey: key.publicKey
)
```

Sighash byte width. `toTxFormat` narrows the sighash to the low 8 bits via `UInt8(nHashType.sighash & 0xff)`. Bitcoin's wire format takes a single byte; passing the full 32-bit sighash word would corrupt every transaction. SwiftBSV enforces this – there's a regression test pinning the byte width.

8.2 Sighash Types

SighashType is a bit-or of a base mode and the FORKID flag. BSV defaults to ALL | FORKID.

SighashType - BSV defaults

```
SighashType.BSV.ALL           // sign every input + every
    output
SighashType.BSV.NONE         // sign every input, no
    outputs
SighashType.BSV.SINGLE        // sign every input, only
    matching output
SighashType.BSV.ALL_ANYONECANPAY // ALL | ANYONECANPAY
```

The FORKID flag was added at the BCH/BSV chain split to prevent transaction replay between BTC and the new chains. Every BSV transaction signature includes it.

8.3 Bitcoin Signed Message (BSM)

BSM is the convention for signing arbitrary text with a Bitcoin private key, returning a Base64-encoded compact signature that any BSM-aware tool can verify against a Bitcoin address.

BitcoinSignedMessage

```
let priv = PrivateKey(network: .mainnet)
let addr = priv.address

let msg = "I authorize transaction abc123..."

// Returns Base64 string
let sig: String = BitcoinSignedMessage.sign(
    message: msg,
    privateKey: priv
)

// Anyone with the address + message + signature can verify
let ok: Bool = BitcoinSignedMessage.verify(
    message: msg,
    signature: sig,
    address: addr
)
```

The verifier never sees the public key directly: it's recovered from the compact signature and matched against the supplied address.

Crash-safety on tampered signatures. Earlier versions of SwiftBSV called `try! Secp256k1.recoverCompact(...)` during verification. A signature with an out-of-range recovery flag (e.g. flipped or truncated by a malicious caller) would crash the host process instead of returning false. The current implementation uses `guard let ... = try? ... else return false` – failure to recover means failure to verify, never a crash.

8.4 Recoverable Signatures and Compression

BSM signatures encode a 1-byte recovery header that selects which of up to four candidate public keys was the signer. Two bits encode parity and the second-derivative branch; one extra bit encodes whether the signer’s public key was compressed when the address was derived. Without the compression bit, verify cannot tell whether the recovered key should be hashed in compressed or uncompressed form, and the resulting address will not match.

SwiftBSV preserves the compression flag across the round trip. If you sign with a compressed-form private key (the SwiftBSV default), the signature header reflects that, and verification against the corresponding compressed-form address succeeds.

ECIES, Electrum/BSV flavour. SwiftBSV implements the Electrum variant of ECIES, identified by the wire prefix "BIE1". This is the same format used by @bsv/sdk's EncryptedMessage class – ciphertext produced by SwiftBSV decrypts on Node.js, and vice versa. Wire-compatible interop is the whole point.

9.1 The BIE1 Wire Format

```
"BIE1" || ephemeralPubKey || ciphertext || HMAC
      4           33           N           32
```

The ephemeral public key is 33 bytes (compressed). The HMAC is 32 bytes (HMAC-SHA-256). The ciphertext is AES-256-CBC over the plaintext, padded with PKCS#7. Total overhead is 69 bytes regardless of message length.

9.2 Key Derivation

The shared secret is the X coordinate of $\text{ephemeralPrivKey} * \text{recipientPubKey}$. From it:

```
key = SHA-512(sharedX)
iv  = key[0:16]      // AES-CBC IV
kE  = key[16:32]     // AES-CBC key (256-bit, 16-byte truncation? no -- 16 bytes!)
kM  = key[32:64]     // HMAC-SHA-256 key
```

16-byte AES key with a 32-byte HMAC key. The Electrum format derives a 128-bit AES key (kE is 16 bytes) and a 256-bit HMAC key. This matches Electrum's historical choice, not the strongest possible parameter set – but it is the format every BSV implementation has shipped. Don't change the slicing; you'll break interop.

9.3 API

ECIESEncryption – encrypt & decrypt

```
import SwiftBSV

let recipient = PrivateKey(network: .mainnet)
let recipientPub = recipient.publicKey

// Sender does not need their own keypair -- ECIES generates an
// ephemeral one per message. Pass nil for senderPrivateKey.
let plaintext: Data = "secret".data(using: .utf8)!
let cipher: Data = try ECIESEncryption.encrypt(
    plaintext: plaintext,
    recipientPublicKey: recipientPub,
    senderPrivateKey: nil
)

// Recipient -- only needs their own private key
let recovered: Data = try ECIESEncryption.decrypt(
    ciphertext: cipher,
    privateKey: recipient
)
// recovered == plaintext
```

9.3.1 Authenticated Sender Mode

Pass a sender private key when you want the recipient to know who sent the message. The protocol still uses an ephemeral key for confidentiality, but the HMAC is keyed in a way the recipient can validate against the named sender.

ECIES – Authenticated

```
let sender = PrivateKey(network: .mainnet)
let cipher = try ECIESEncryption.encrypt(
    plaintext: data,
    recipientPublicKey: recipientPub,
    senderPrivateKey: sender
)

// Recipient supplies their own private key + the expected
// sender public key. HMAC failure means either the message
// was tampered or the sender wasn't who you expected.
let recovered = try ECIESEncryption.decrypt(
    ciphertext: cipher,
    privateKey: recipient,
    senderPublicKey: sender.publicKey
)
```

9.4 Errors

ECIESError is a small enum:

- `.invalidPublicKey` – the embedded sender public key didn’t deserialize as a valid curve point
- `.hmacMismatch` – ciphertext failed authentication
- `.invalidCiphertextFormat` – wrong magic prefix, truncated input, or non-canonical encoding
- `.encryptionFailed` / `.decryptionFailed` – AES-CBC failure (typically padding)

Constant-time HMAC comparison. The library uses `Data’s ==` for HMAC verification, which short-circuits on the first mismatched byte. For a network protocol that’s reachable by an unbounded attacker, you would prefer a constant-time comparison. SwiftBSV’s threat model is local: the ciphertext has already been received, the attacker cannot mount a real-time timing oracle, and the cost-benefit of constant-time comparison doesn’t move. If you embed SwiftBSV in a server-side application, audit this assumption against your own threat model.

10.1 Script Anatomy

A Bitcoin Script is a sequence of Chunks – each chunk is either a single opcode or a PUSHDATA containing literal bytes. SwiftBSV models the structure faithfully:

Script – Standard P2PKH

```
let priv = PrivateKey(network: .mainnet)
let addr = priv.address

// Build by hand
var script = Script()
_ = script
    .append(.OP_DUP)
    .append(.OP_HASH160)
    .appendData(Crypto.sha256ripemd160(priv.publicKey.toBuffer()))
    .append(.OP_EQUALVERIFY)
    .append(.OP_CHECKSIG)

// Or use the convenience
let sameScript = addr.toTxOutputScript()
```

10.2 Opcode Catalogue

Opcodes live in Sources/SwiftBSV/Scripts/OP_CODE/ grouped by category:

- **Push Data** – OP_FALSE/0, OP_PUSHDAT1/2/4, OP_1NEGATE, OP_1...OP_16
- **Flow Control** – OP_NOP, OP_IF, OP_NOTIF, OP_ELSE, OP_ENDIF, OP_VERIFY, OP_RETURN
- **Stack** – OP_DUP, OP_DROP, OP_SWAP, OP_OVER, OP_ROT, OP_PICK, OP_ROLL, OP_DEPTH...
- **Splice** – OP_CAT, OP_SPLIT, OP_NUM2BIN, OP_BIN2NUM, OP_SIZE
- **Bitwise Logic** – OP_INVERT, OP_AND, OP_OR, OP_XOR, OP_EQUAL, OP_EQUALVERIFY
- **Arithmetic** – OP_ADD, OP_SUB, OP_MUL, OP_DIV, OP_MOD, OP_BOOLAND, OP_BOOLOR...

- **Crypto** – OP_HASH160, OP_HASH256, OP_SHA1, OP_SHA256, OP_RIPEMD160, OP_CHECKSIG, OP_CHECKMULTISIG
- **Lock Time** – OP_CHECKLOCKTIMEVERIFY, OP_CHECKSEQUENCEVERIFY
- **Reserved Words** – the Chronicle-restored opcodes that BSV brought back from the original protocol set

The full enumeration is exhaustive – BSV restored every opcode that BTC removed at the chain split. The library exposes them all as enum cases on `OpCode`.

Script vs FORTH. Henceforth's FORTH terminal is the human-facing interface to script construction – you type `OP_DUP OP_HASH160 ... OP_CHECKSIG` and the recorder emits the bytes. SwiftBSV's `Script` type is the programmatic interface, used by `TxBuilder` and by anyone embedding script construction inside a Swift host. The two represent the same underlying objects; pick the one that matches your context.

11.1 Transaction Structure

A BSV transaction is the same algebraic object the white paper describes:

```
Transaction
  +-- version          (4 bytes)
  +-- inputs           (varint count + N inputs)
  |   \-- (prevTxid, prevVout, scriptSig, sequence)
  +-- outputs          (varint count + M outputs)
  |   \-- (value, scriptPubKey)
  \-- lockTime         (4 bytes)
```

SwiftBSV exposes `Transaction`, `TransactionInput`, and `TransactionOutput` as value types. You can construct one by hand, but the `TxBUILDER` fluent API is the conventional path.

11.2 TxBuilder – Fluent Construction

`TxBUILDER` is a class (not a struct) because building a transaction is a sequence of mutations against an evolving state, and a class lets you chain calls that all return `self`.

TxBUILDER – A complete pay-to-address transaction

```
let alice = PrivateKey(network: .mainnet)
let bob   = Address(fromString: "1A1zP1...", network: .mainnet)!

let utxo: TransactionOutput = ... // from your UTXO source
let prevTxid: Data          = ...
let prevVout: UInt32        = 0

let builder = TxBUILDER()

_ = builder
  .setVersion(2)
  .setFeePerKb(50)
  .setChangeAddress(alice.address)
```

```

        .inputFromPubKeyHash(
            txHashBuffer: prevTxid,
            txOutNum: prevVout,
            txOut: utxo,
            pubKey: alice.publicKey
        )
        .outputToAddress(value: 50_000, address: bob)

let built = try builder.build(useAllInputs: true)

// Sign every input
for i in 0..<built.transaction.inputs.count {
    _ = built.signInTx(
        nIn: i,
        privateKey: alice
    )
}

let raw: Data = built.transaction.serialized()
let txid: String = built.transaction.txID

```

11.2.1 Method Reference

TxBUILDER – Setters & inputs

```

.setVersion(_ version: UInt32)
.setNLockTime(_ nLockTime: UInt32)
.setFeePerKb(_ fee: Float)
.setChangeAddress(_ changeAddress: Address)
.setChangeScript(_ changeScript: Script)

.inputFromScript(
    _ txHashBuffer: Data,
    txOutNum: UInt32,
    txOut: TransactionOutput,
    script: Script,
    nSequence: UInt32
)

.inputFromPubKeyHash(
    txHashBuffer: Data,
    txOutNum: UInt32,
    txOut: TransactionOutput,
    pubKey: PublicKey,
    nSequence: UInt32 = 0xffffffff,
    nHashType: SighashType = SighashType.BSV.ALL
)

.outputToAddress(value: UInt64, address: Address)
.outputToScript(value: UInt64, script: Script)

```

11.2.2 `build(useAllInputs:)`

`build(useAllInputs:)` computes a final fee against `feePerKb`, picks UTXOs to cover the outputs, and emits a change output if there's leftover value. Pass `true` to force every added input into the final transaction (useful when you have already pre-selected coins and want no further filtering); pass `false` to let the builder pick the minimum subset.

11.2.3 `signInTx(nIn:privateKey:)`

Signs a single input with the supplied private key. Repeat for every input. The signature uses `SighashType.BSV.ALL | FORKID` by default; pass an explicit `sighashType:` to override.

Build, then sign – always in that order. `build` computes input/output structure and locks down the transaction shape. Signing earlier produces signatures over a transaction that no longer exists by the time `build` mutates the input set. The wallet code in `Henceforth` follows the build-then-sign discipline; replicate it in your own consumers.

What SwiftBSV does in SPV. The library implements the verification side: given a block header, validate its proof-of-work; given a chain of headers, validate that each links to its parent; given a Merkle proof, validate that a transaction belongs to a header's Merkle root. The library does not fetch headers or proofs – that's the consumer's job. Henceforth's HeaderSyncService and SPVService are reference consumers; see Henceforth's SPV chapter for the architectural pattern.

12.1 BlockHeader Protocol

Anything that can answer the eight standard fields conforms to the BlockHeader protocol. You don't have to use SwiftBSV's own model type – WhatsOnChain's WOCGetHeadersModel (in Henceforth) conforms directly:

BlockHeader protocol

```
public protocol BlockHeader {
    var hash: String { get }
    var height: Int { get }
    var version: Int { get }
    var merklerooroot: String { get }
    var time: Date { get }
    var nonce: Int { get }
    var bits: String { get }
    var previousblockhash: String { get }
}
```

12.2 BlockHeaderValidator

The validator runs three checks:

BlockHeaderValidator

```
let validator = BlockHeaderValidator()
```



```
// 1. Single-header proof-of-work
let powOK: Bool = validator.validateProofOfWork(header: header)

// 2. Chain-link to parent
let linkOK: Bool = validator.validateChainLink(
    header: child,
    previousHeader: parent
)

// 3. Full chain -- sorts by height, validates PoW + linking
let result: ChainValidationResult =
    validator.validateHeaderChain(headers: headers)

if result.isValid {
    print("validated \(result.validatedCount) headers")
} else {
    print(result.errorMessage ?? "unknown error")
}
```

12.2.1 Block Hashing

Computing a block's hash is the same procedure the white paper specifies:

BlockHeaderValidator.calculateBlockHash

```
// Serialize the 80-byte header (version + prev hash + merkle root
// + timestamp + bits + nonce, with the hash and merkle root
// reversed to wire byte order)
let serialized: Data? = validator.serializeBlockHeader(header: h)

// Double-SHA-256 and reverse to display byte order
let hash: String? = validator.calculateBlockHash(header: h)
```

12.2.2 Target from Bits

The bits field encodes the difficulty target in compact form. The validator unpacks it for PoW comparison:

Target arithmetic

```
let target: UInt256? = validator.targetFromBits(bits: header.bits)

// PoW passes if computed hash, interpreted as UInt256,
// is strictly less than target
let hashValue: UInt256? = UInt256(hexString: computedHash)
let powOK = hashValue! < target!
```

UInt256 is a custom big-endian 256-bit comparable struct. It exists specifically for hash/target comparisons and has no arithmetic beyond left-shift and ordering – enough to evaluate coefficient $\ll ((\text{exponent} - 3) * 8)$.

12.3 Merkle Proof Formats

SwiftBSV understands three Merkle proof formats:

- **Standard MerkleProof** – generic representation: (txid, blockHash, index, nodes).
- **TSC (BRC-74) TSCMerkleProof** – the Technical Standards Committee compact format used by BSV miners and indexers.
- **WhatsOnChain WOCMerkleProof** – the WoC API response shape with directional branches ("L" / "R").

MerkleVerifier – Standard proof

```
let proof = MerkleProof(
  txid: "...",
  blockHash: "...",
  blockHeight: 800_000,
  merkleRoot: "...",
  index: 42,
  nodes: ["aaaa...", "bbbb...", ...]
)

let verifier = MerkleVerifier()
let ok = verifier.verifyMerkleProof(
  proof,
  expectedMerkleRoot: header.merkleroot
)
```

MerkleVerifier – TSC (BRC-74)

```
let tsc = TSCMerkleProof(
  index: 42,
  txOrId: "...",
  target: header.merkleroot,
  nodes: ["...", "..."],
  targetType: "merkleroot",
  proofType: "branch",
  composite: false
)

let ok = verifier.verifyTSCProof(
  tsc,
  expectedMerkleRoot: header.merkleroot
)
```

MerkleVerifier – WhatsOnChain

```
// Verify the WoC proof against a header's merkle root that you
// fetched independently. NEVER trust the proof's embedded root --
```

```
// always cross-check with a header from the chain-validated store.
let ok = verifier.verifyWOCProofAgainstRoot(
    wocProof,
    expectedMerkleRoot: localHeader.merkleroot
)
```

12.3.1 Pair Hashing in Display vs Wire Order

The trickiest part of Merkle verification is the byte-order convention. Bitcoin hashes appear in the API and the UI in display order (most significant byte first), but the underlying double-SHA-256 operates on wire order (least significant byte first). `doubleSHA256HashPair` encapsulates this:

doubleSHA256HashPair

```
public func doubleSHA256HashPair(left: String, right: String) ->
String {
    let leftBytes = Data(hex: left)
    let rightBytes = Data(hex: right)

    var combined = Data(leftBytes.reversed())
    combined.append(Data(rightBytes.reversed()))

    let hash1 = Data(combined).sha256()
    let hash2 = Data(hash1).sha256()

    return Data(Data(hash2).reversed()).hex
}
```

The two reversals – on input and on output – mean the function takes display-hex in and produces display-hex out, while internally hashing in wire order. Get this wrong (forget either reversal, or do both reversals on only one side) and the entire Merkle path will appear to mismatch the root for opaque reasons.

Defense in depth. Even when verifying a Merkle proof against a header’s merkle root, you should validate the header’s PoW before accepting the proof. A header with a wrong merkle root could be served to you by a hostile API; an attacker who controls one endpoint cannot also produce a valid PoW pre-image. Always sequence: (1) PoW-validate the header, (2) verify the Merkle proof against the validated header’s root, (3) only then declare the transaction confirmed.

12.4 SPV Errors

`SPVError` is the closed enum returned by verification failures:

- `.invalidMerkleProof` – the proof structure is malformed (e.g. wrong number of branches for the index)

- `.blockHeaderNotFound` – the consumer was asked to verify against a header it doesn't have
- `.merkleRootMismatch` – the proof verified to a different root than the header
- `.invalidProofFormat` – format-level rejection
- `.networkError(String)` – carried up from the consumer's network layer
- `.txNotInBlock` – proof unavailable for the transaction (typical when a tx is mempool-only)
- `.invalidBlockHash` – the block hash failed PoW or didn't link
- `.headerChainNotSynced(requiredHeight: Int, localTip: Int)` – the tx is in a block above the locally validated tip; the consumer should retry once sync catches up, not present this as a verification failure

The distinction between `.blockHeaderNotFound` and `.headerChainNotSynced` is what lets the consumer (Henceforth) show "△ SPV PENDING" for sync gaps and "× SPV FAILED" for genuine verification failures.

13.1 SwiftPM

In your `Package.swift`:

Package.swift – Adding SwiftBSV

```
// swift-tools-version:5.9
import PackageDescription

let package = Package(
    name: "MyApp",
    platforms: [.iOS(.v16), .macOS(.v13), .tvOS(.v16)],
    dependencies: [
        .package(
            url: "https://github.com/henryhudson/SwiftBSV",
            branch: "main"
        )
    ],
    targets: [
        .target(
            name: "MyApp",
            dependencies: ["SwiftBSV"]
        )
    ]
)
```

13.2 Xcode

File → Add Packages... → paste the GitHub URL → Branch: main. Add SwiftBSV to your application target's framework list. Done.

13.3 Platform Requirements

- **Swift tools 5.9+** (the `Package.swift` declares this minimum)

- iOS 16 / macOS 13 / tvOS 16 or later
- Xcode 16+ for full Swift 6 strict-concurrency builds

13.4 Importing

```
import SwiftBSV
```

```
import SwiftBSV

// Everything documented in this book is in the SwiftBSV module.
// No subspec or qualified import -- single import covers all
// public surface.
```

13.4.1 Swift 6 Concurrency Notes

Every public type that crosses an actor boundary in real code is `Sendable`. You can pass `PrivateKey`, `PublicKey`, `Address`, `Bip32`, `Signature`, `MerkleProof`, and `ChainValidationResult` freely between `@MainActor` and detached `Task` contexts.

A small number of types remain non-`Sendable` because mutation is part of their interface:

- `TxBuilder` – holds an evolving `Transaction`; mutate it on one isolation domain and pass the resulting `Transaction` (which is `Sendable`) across boundaries.
- `Bip39` – a static-only entry point in practice; the class itself is not constructible.

No `@preconcurrency import SwiftBSV` required. Earlier Henceforth code carried `@preconcurrency import` on five files because not every type was `Sendable`. The current build is clean – those workarounds were removed once the underlying types were marked. If you find yourself needing `@preconcurrency` against `SwiftBSV`, file an issue: it's a bug in the library, not your consumer.

13.5 License

`SwiftBSV` is published under the same license as the upstream `Yenom BitcoinKit` it forked from – see `LICENSE` in the repository root. The fork preserves `Yenom`'s copyright notices on files that retain their original authorship and adds `Henry Hudson`'s copyright on materially rewritten files.

13.6 Contributing

The repository accepts pull requests for:

- Bug fixes against documented behavior
- New tests – particularly in the SPV, ECIES, and Type42 areas where coverage is shallower than ECDSA
- Documentation corrections – this book is generated from `swiftbsv.tex` in the LaTeX repo; corrections welcome there too
- Performance improvements with benchmarks

API additions are accepted on a case-by-case basis – the goal is a small library, not a kitchen sink. Open an issue describing the use case before opening a PR for a new public type.